

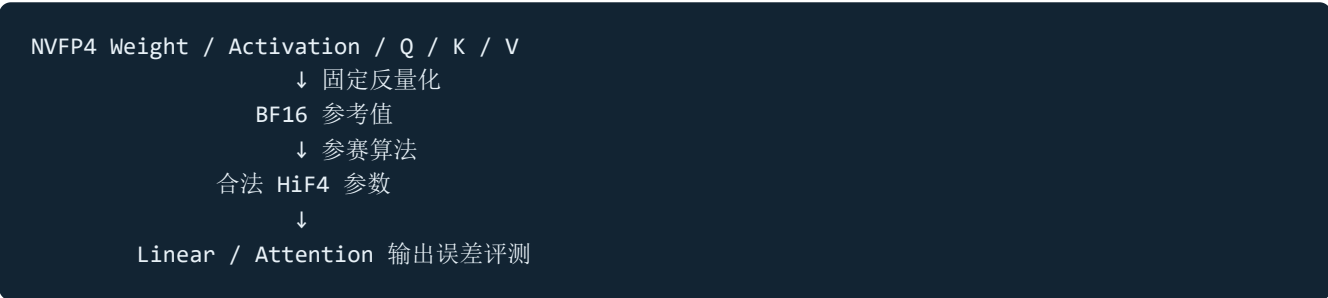
NVFP4 → HiF4 项目调研与方案总结

项目：2026 华为算法大赛初赛 - 高精度 4Bit 数值转换算法

更新时间：2026-08-31

1. 项目目标

本项目需要将 NVIDIA NVFP4 数据转换成昇腾 HiF4 参数，并尽量保持大模型 Linear 和 Attention 的计算结果不变。



比赛参考值是给定 NVFP4 数据反量化后的结果，不是模型最初的 BF16/FP32 数据。因此，本质上是一次受 HiF4 格式约束的二次量化。

单个用例得分为：

$$\text{Score} = (\text{MSE_STD} - \text{MSE_PLAYER}) / \text{MSE_STD}$$

参赛算法误差低于标准 HiF4 算法时得正分，高于标准算法时得负分。

2. 为什么需要这种转换

FP4 可以降低大模型的权重、Activation 和 KV Cache 存储及搬运开销。但不同硬件平台采用的 FP4 格式并不一致：

特征	NVFP4	HiF4
元素格式	E2M1	S1P2
block 大小	16	64
局部 scale	E4M3	两层 {1,2} 微尺度
基础 scale	FP32 per-tensor	E6M2 per-64
非负元素值	0、0.5、1、1.5、2、3、4、6	0~1.75，步长 0.25

NVFP4 的四个 16 元素 block 合并成一个 HiF4 64 元素 block 后，原有 scale 无法直接复制。算法必须重新选择 E6M2、两级微尺度和 mant。

该项目面向的实际价值是：减少模型跨硬件平台部署时重新训练或重新量化的成本，并提升既有 NVFP4 模型资产在 HiF4 平台上的兼容性。

3. 赛题任务

3.1 Linear

输入：

- NVFP4 Weight;
- Calibration Activation;
- 在线 Test Activation。

目标：

$$\min \text{MSE}(X_{\text{HiF4W_HiF4}}(T), X_{\text{NVFP4W_NVFP4}}(T))$$

校准函数负责量化 Weight 并生成 `activation_state`；动态函数使用 state 量化当前 Activation。

3.2 Attention

输入：

- Calibration Q/K/V;
- Test Q/K/V;
- Q head、KV head 和 head dimension。

目标：

$$\min \text{MSE}(\text{Attention}(Q_{\text{HiF4}}, K_{\text{HiF4}}, V_{\text{HiF4}}), \text{Attention}(Q_{\text{NVFP4}}, K_{\text{NVFP4}}, V_{\text{NVFP4}}))$$

校准函数生成 `q_state`、`k_state` 和 `v_state`，在线阶段分别量化 Q、K、V。

3.3 关键约束

- 必须实现任务书规定的六个接口；
- HiF4 参数必须严格合法；
- state 只能包含基础类型、CPU Tensor、list/tuple 和字符串键 dict；
- 单次提交总运行时间不超过五分钟；
- 禁止计算 `A @ W` 后利用结果反向拟合 `Q(A)`；
- `self_check.py` 只检查格式，不代表精度得分。

4. 固定的数值规则

4.1 NVFP4 反量化

比赛已经确定 NVFP4 反量化方法：

```
x = quant_float.unflatten(-1, (-1, 16))
x = x * scale_float.unsqueeze(-1)
x = x.flatten(-2, -1).to(torch.bfloat16)
```

算法不需要解析原始 4-bit bitstream，也不能改变判题器对 NVFP4 的解释。

4.2 HiF4 反量化

HiF4 每 64 个元素包含：

- 1 个 E6M2 `scale_factor`；
- 8 个 `scale_lv2`，每个控制 8 个元素；
- 16 个 `scale_lv3`，每个控制 4 个元素；
- 64 个 `sign/mant`。

固定反量化公式：

$$\hat{x} = \text{sign} \times \text{mant} \times \text{scale_lv3} \times \text{scale_lv2} \times \text{scale_factor}$$

合法范围：

```
scale_factor: E6M2
scale_lv2    : 1 或 2
scale_lv3    : 1 或 2
sign         : -1、0 或 1
mant         : 0~1.75, 步长 0.25
```

反量化规则是确定的；比赛需要优化的是如何选择这些参数。

5. 当前 baseline 的方案

当前 `baseline.py` 使用“格式内搜索 + 校准二阶补偿”的两层结构。

5.1 第一层：HiF4 格式内搜索

`_encode()` 的流程是：

1. 将四个 NVFP4 block 合并为 64 元素；
2. 从 `max_abs/7` 附近选取多个 E6M2 候选；
3. 在 HiF4 树状约束下选择 lv2/lv3；
4. 将每个元素舍入到 S1P2；

5. 对给定离散参数重新拟合连续最优 scale;
6. 将 scale 舍入到最近合法 E6M2;
7. 选择重构 MSE 最小的候选。

这比官方单纯由局部最大值决定 scale 的方式更强，但目前搜索范围较窄，scale 回归后也没有再次更新 mant。

5.2 第二层：SVD 二阶统计

Linear 中，设权重量化误差为：

$$D = \hat{W} - W$$

校准输入为 A ，最终输出误差为：

$$\|AD^T\|_F^2 = \sum_r d_r^T (A^T A) d_r$$

因此不同通道的误差重要性由 $A^T A$ 决定。baseline 使用截断 SVD 近似该矩阵：

$$A \approx U_k \Sigma_k V_k^T$$

$$A^T A \approx V_k \Sigma_k^2 V_k^T$$

这样可以用 `[rank, hidden_size]` 的低秩 state 表示重要方向，避免保存完整 Hessian。

5.3 Schwarz 分组舍入

普通最近邻对每个元素独立选择 floor 或 ceil。当前方案每次联合枚举一组元素的上下舍入组合，并根据全局投影误差更新后续决策。

它允许以下现象：某个元素不选择离自身最近的值，但它产生的误差与其他通道误差抵消，最终矩阵乘误差反而更小。

这与 GPTQ 的二阶误差补偿思想一致。

6. 外部分享观点的分析

本节内容来自他人分享，不属于当前 baseline 的既定方案。这里仅分析其数学含义、适用边界和可能的验证价值，不直接将其列为实现要求。

6.1 Softmax 具有行平移不变性

对任意常数 c ：

$$\text{softmax}(z+c1) = \text{softmax}(z)$$

因此 Q/K 量化造成的 logits 误差中，**每一行的常数偏移不会影响 softmax 输出。**

若参考 logits 为 L ，量化误差为 $E = \hat{L} - L$ ，直接最小化：

$$\|E\|_F^2$$

会把无害的行常数偏移也计入误差。更合理的代理目标是先去掉每行均值：

$$E_c = E - \text{mean}(E, \text{dim}=-1)$$

再最小化：

$$\|E_c\|_F^2$$

更精确的局部目标可使用 softmax Jacobian：

$$J(p) = \text{diag}(p) - pp^T$$

常数方向满足 $J(p)1=0$ ，说明它确实是 softmax 的零敏感方向。

这说明“原始 logits MSE”不完全等价于 softmax 输出误差。行中心化 logits MSE 可以作为补充诊断指标，但它是否适合作为本项目的优化目标仍需实验确认。

还要注意：量化 Q/K 后产生的误差通常不是任意可控的行常数。平移不变性说明某类误差无害，但不代表算法可以自由把其他误差转换成行常数，因此不能仅凭这一性质认定一定能提高得分。

6.2 Softmax 每行和恒等于 1

设 Attention 权重为：

$$P = \text{softmax}(L)$$

则：

$$P1=1$$

Attention 输出为：

$$O = PV$$

V 量化误差为 ΔV 时：

$$\Delta O = P \Delta V$$

每个输出 token 都是 V 误差的加权平均。因此：

- 被高 Attention 权重访问的 token 更重要；
- V 的普通 Tensor MSE 不等于最终输出 MSE；

- 若所有 V token 存在相同偏移 b ，则 $P(\Delta V+b)=P\Delta V+b$ ，该偏移不会被抵消；
- V 需要关注 Attention 权重统计，而不是简单复用 Q/K 的通道协方差目标。

如果后续要验证这一观点，可以比较如下 calibration 代理：

$$\min \sum_s \|P_s(V_s - \hat{V}_s)\|_F^2$$

但在线接口不能同时看到完整 Q/K/V，如何把 P 的统计压缩成可泛化的轻量 `v_state` 并不明确。因此这首先是一个值得验证的分析方向，而不是当前已经确定的 V 方案。

7. 对 “GPTQ 做两倍” 的理解

“GPTQ 做两倍” 这句话本身存在歧义，至少可能有三种含义：

1. GPTQ 或二阶补偿执行两遍；
2. 补偿项乘以 2；
3. 相比某个基线使用两倍的 block、rank 或搜索预算。

在没有更多上下文时，不能确定分享者具体指哪一种，也不应直接写入当前方案。

如果它指 “执行两遍”，可能对应以下双阶段过程：

阶段一：格式参数优化

- 搜索 E6M2、lv2、lv3；
- 以普通重构 MSE 或加权 MSE 得到稳定初值；
- 确定每个元素的 floor/ceil 候选。

阶段二：GPTQ/Schwarz 误差补偿

- 第一遍按重要性处理通道；
- 更新累计投影误差；
- 第二遍重新检查所有 floor/ceil 决策；
- 只接受能降低全局二阶目标的翻转。

当前 baseline 本身已经是 “先 `_encode()`，再进行 Schwarz 多轮更新”，与两遍二阶优化有一定相似性。但这只能说明概念上可能相关，不能证明就是分享者所说的 “两倍”。

如果它指 “补偿项乘 2”，则需要谨慎：GPTQ 的补偿系数来自二阶近似或逆 Hessian，直接乘 2 会改变原优化问题，可能造成过补偿和不稳定，目前没有充分依据支持。

因此，这条分享更适合作为待澄清、待消融的实验备注：

需要记录：

```
0 pass: 纯格式搜索
1 pass: 一次 GPTQ/Schwarz
2 pass: 两次 GPTQ/Schwarz
3 pass: 检查是否继续有收益
```

在明确原意之前，不将“两倍”纳入正式实现路线。

8. 当前方案的主要问题

8.1 接口尚未闭环

当前 baseline 不是可直接提交的 `solution.py`：

- Weight calibration 返回格式不符合任务书；
- 缺少正式的动态 Activation/Q/K/V 接口；
- 没有完整 `activation_state` 路径。

8.2 Schwarz 对角项需要核对

若目标是“低秩非对角项 + 完整精确对角项”，建议使用：

$$H_{\text{approx}} = V_w^T V_w + \text{diag}(\text{an_full} - \text{an_svd})$$

当前实现可能把低秩部分对角线重复计算，应做消融验证。

8.3 GQA 的 Q 聚合方式可能失真

一个 KV head 对应多个 Q head 时，合理敏感度为：

$$H_K = \sum_h Q_h^T T Q_h$$

当前方案先对多个 Q head 求平均，可能因正负抵消低估重要方向。建议把 Q heads 沿样本维堆叠。

8.4 V calibration 缺少 Attention 目标

当前使用 `SVD(V)` 指导 V 舍入，但最终误差由 $P \Delta V$ 决定。应比较：

1. 普通 V 重构 MSE；
2. `SVD(V)`；
3. 基于 calibration Attention 权重的 V 重要性。

8.5 时间风险

完整 SVD、多候选搜索和多轮 Schwarz 都会增加 CPU 时间。需要区分：

- Weight：离线计算，可使用较强搜索；
- Activation/Q/K/V：在线调用，应限制候选、rank 和迭代次数。

9. 当前方案与待验证观点

9.1 当前应推进的确定方案

1. 补齐六个接口并通过 self-check;
2. 固定 NVFP4 反量化;
3. 搜索 $\max/7$ 附近的 E6M2;
4. 精确选择合法 lv2/lv3;
5. 回归 scale 后重新更新一次 mant;
6. 记录 Linear/Attention MSE 与耗时。

Linear 的当前核心仍是：

1. Weight 使用低秩二阶统计;
2. 修正并验证 Hessian 对角残差;
3. 补齐动态 Activation 路径;
4. 通过消融确认 SVD/Schwarz 是否稳定优于普通搜索。

Attention 的当前核心仍是：

1. Q 使用 K 的统计;
2. K 使用所有对应 Q heads 的统计，而不是简单均值;
3. 分别验证 Q、K、V 的实际输出误差;
4. 控制 dynamic 阶段的运行成本。

9.2 外部分享带来的待验证问题

以下内容只列为实验问题，不是当前方案：

1. 行中心化 logits MSE 是否比普通 logits MSE 更能预测最终 Attention MSE?
2. softmax Jacobian 加权是否带来稳定收益，还是过拟合 calibration?
3. 利用 softmax 行和为 1 构造的 V 权重是否能够压缩为可泛化 state?
4. “GPTQ 做两倍”的原意究竟是两遍优化还是补偿放大?
5. 若执行两遍，第二遍的精度收益是否值得额外时间?

这些问题应与当前 baseline 分开做消融，验证有效后再决定是否进入正式方案。

10. 实验矩阵

编号	格式搜索	二阶优化	Q/K 目标	V 目标
A0	官方 max-based	无	Tensor MSE	Tensor MSE
A1	E6M2 + lv2/lv3 搜索	无	Tensor MSE	Tensor MSE
A2	同 A1	1 遍	logits MSE	Tensor MSE
A3	同 A1	2 遍	logits MSE	Tensor MSE
A4 (外部观点验证)	同 A1	1 遍	行中心化 logits MSE	Tensor MSE
A5 (外部观点验证)	同 A1	1 遍	行中心化/Jacobian	Attention-aware V

每项至少记录：

```
格式合法性
Tensor 重构 MSE
Linear 输出 MSE
Attention logits MSE
行中心化 logits MSE
Softmax 输出 MSE
Attention 最终输出 MSE
Calibration/Dynamic 时间
State 大小
```

11. 当前结论

当前最有价值的工作是先把已有 baseline 变成清晰、可提交、可验证的闭环：

```
HiF4 合法搜索
+
Linear 二阶误差补偿
+
Q/K 正确的交叉统计
+
完整的动态量化接口
+
时间与精度消融
```

外部分享的两个 softmax 性质在数学上成立：

- 1. 行平移不变性说明 Q/K 不必惩罚 logits 的行常数误差；
 - 2. 每行和为 1 说明 V 输出是加权平均，但所有 token 的共同偏移会完整保留。
- 但它们目前只提供了新的误差分析视角，尚未证明能够直接转化为比赛收益。“GPTQ 做两倍”含义不明确，也暂不纳入确定方案；需要在获得更多上下文或完成独立消融后再判断。

12. 资料

1. [比赛中文任务书](#)
2. [HiFloat4 Format for Language Model Inference](#)
3. [HiFloat4 官方说明](#)
4. [NVIDIA NVFP4 官方文档](#)
5. [Pretraining Large Language Models with NVFP4](#)
6. [GPTQ](#)
7. [SmoothQuant](#)
8. [AWQ](#)
9. [QuaRot](#)
10. [SpinQuant](#)
11. [KIVI](#)